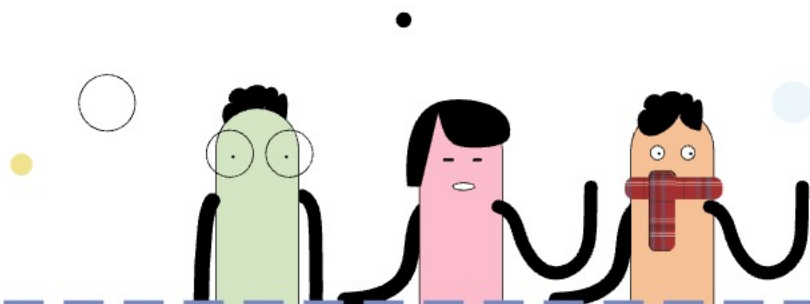
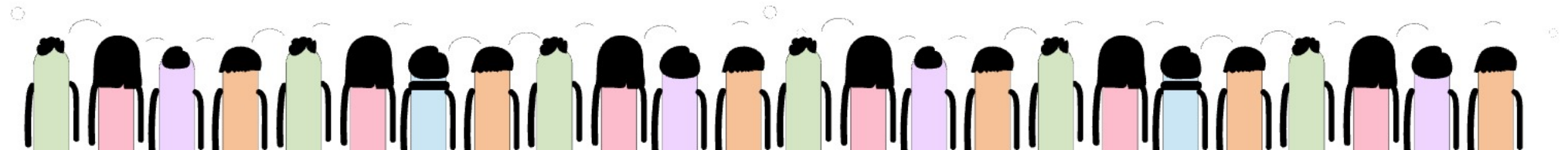
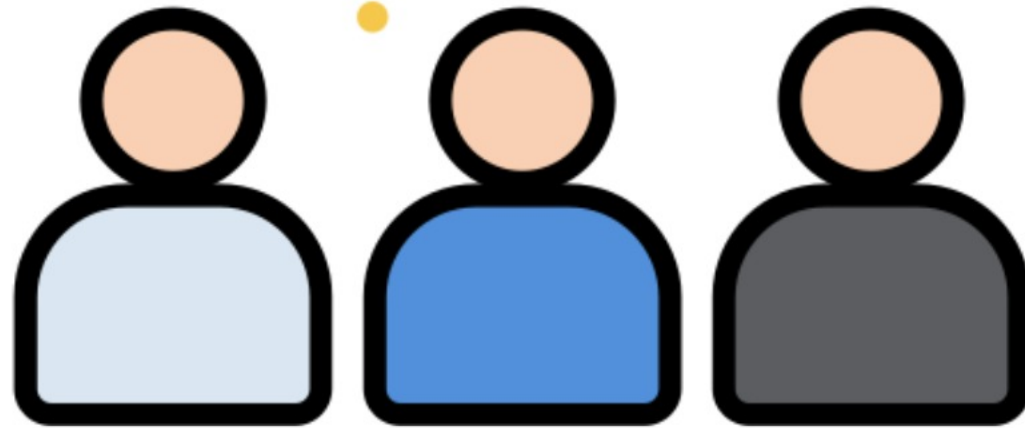




Claude

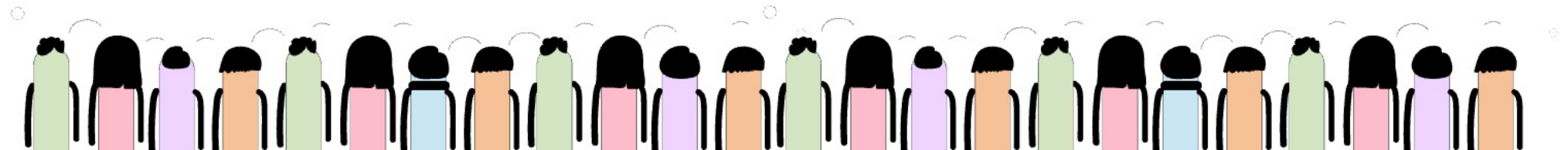
Projects



Project1

we need summary what all steps we did in this project in
projectexecutionsteps .md

give me jpeg image what all did the participantsv learn
here in two images from first prompt tille end or in
form of the slides



Diagnose & Fix



DOMAIN: DAILY INTEREST ACCRUAL

ROUNDING CHANGE = REGULATED BEHAVIOR

SYMPTOM • NIGHTLY RECON FAILURE

Ledger total $\neq$ Σ per-account interest

Gap: a few paise • Scale: ~50,000 accounts

INVESTIGATION — TWO PATHS TRACED, LINE BY LINE

AccrualService
.ComputeDailyAccrual

● CONFIRMED

principal & rate cast to **double**, multiplied, cast back, rounded once.**PostingService**
.ComputeLedgerBatchTotal

● CONFIRMED

re-sums rounded entries via a second **double** accumulator, rounds *again*.**Root cause:** both paths secretly round-tripped through double before landing back in decimal — and the batch total was rounded **twice**.

APPROVED CONSTRAINTS — SIGNED OFF BEFORE ANY CODE CHANGED

- ✓ decimal only — no double/float
- ✓ `MidpointRounding.AwayFromZero`
- ✓ round once, per account — never round the batch total
- ✓ no changes to rates / schedule / recon job
- ✓ future accruals only — no backfill

TEST-DRIVEN — RED SEEN BEFORE GREEN

RED

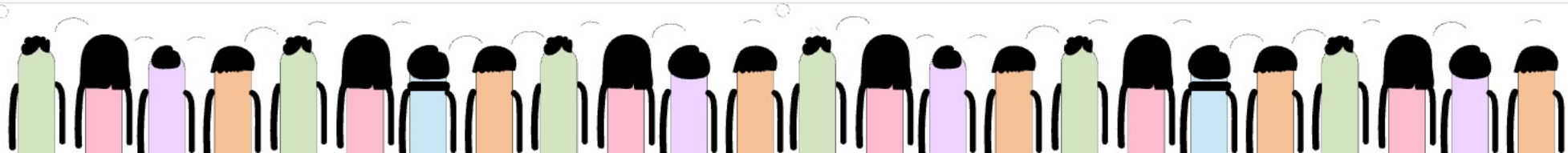
2 tests fail pre-patch

GREEN

9 / 9 pass post-patch

0.00

gap, 5,000-account fixture

Lesson: never mix double/float into decimal money math. Never round the same value twice.**Lesson:** "tests pass" is not proof — demand a numeric acceptance criterion, and see the failure before the fix.



Trust, but Verify



SELF-REVIEW — THE AI FLAGGED ITS OWN DIFF

- 01 Stale comments still describe the old, now-fixed buggy behavior as current
- 02 Two summation methods are now functionally duplicate — no consolidation call made
- 03 No test covers an exact rounding-midpoint case under the new mode

💬 "As an AI coder, I don't trust it — we need to track it."

AI CHANGE LOG — NUMBERED, VERIFIABLE

T1 AccrualService — before/after, why, verified-by **T2** PostingService — before/after, why, verified-by

+ explicit list of everything NOT touched: rates, schedule, models, recon job, no backfill

Lesson: AI-authored changes need an audit trail a human can independently spot-check — not a narrative summary.

GENERALIZED — BUILT ONCE, REUSABLE ON ANY .NET PROJECT



dotnet-code-review skill

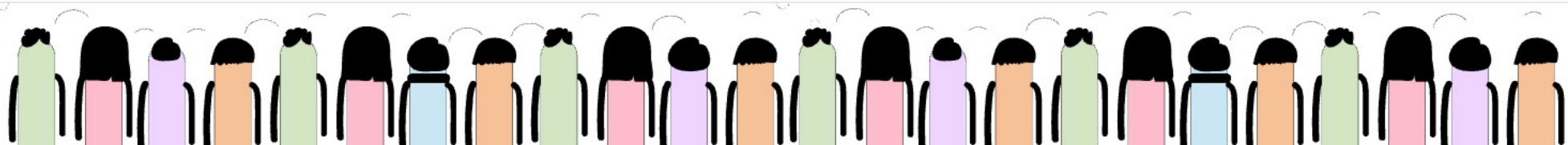
numeric-type audit · test-evidence rigor · scope discipline · AI-change verification



re-applied, same session

run against this project again — re-confirmed the same 3 open items, no new surprises

FINAL TAKEAWAY — THE FULL LOOP





< > interest-accrual-lab

Name

InterestAccrualLab.sln

> src

README.md

> tests

CODE_REVIEW.md

AI_CHANGE_LOG.md

▼ reports

recon-fail-2026-08-16-investigation.md

04-implement.md

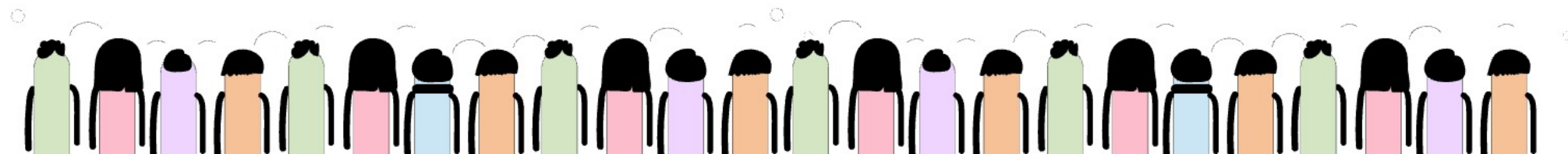
05-validate.md

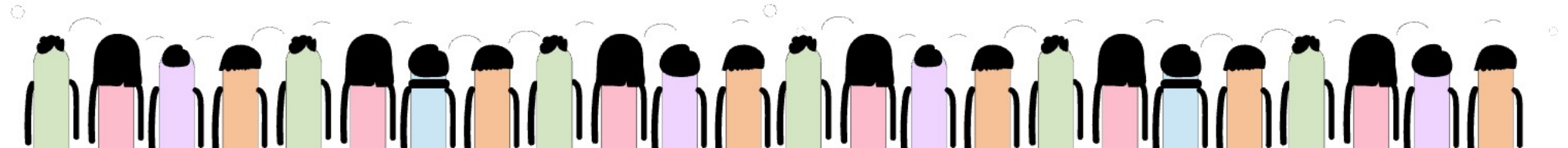
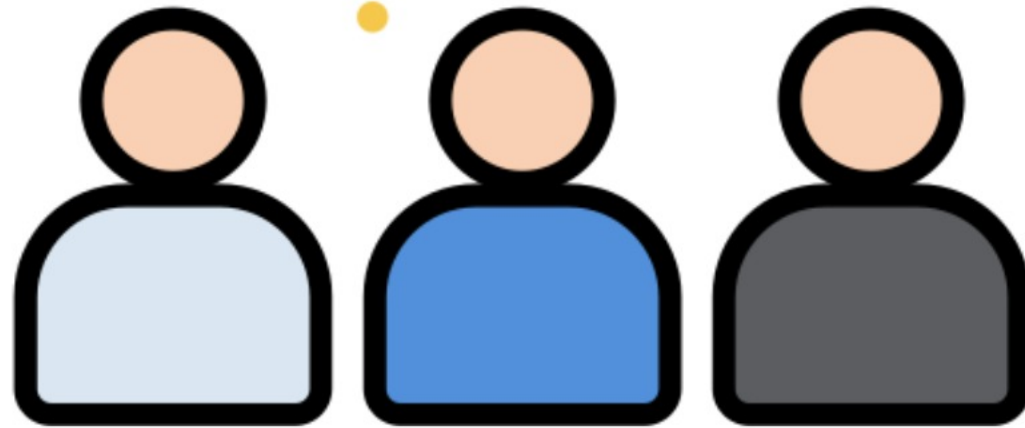
06-review.md

07-report.md

08-skill-code-review.md

PROJECT_EXECUTION_STEPS.md





Project2

SLIDE 1 OF 2 – DIAGNOSIS & VERIFICATION

Finding and fixing the double-booking race

T1 – T3

T1 Diagnosed the race, fixed it at the database

`bookSeat()` did check-then-act: `isAvailable()` and `insert()` ran on separate connections with a 150ms pricing call between them — wide enough for two requests to both pass the check. Added a `UNIQUE(trip_id, seat_no)` constraint so the database, not the earlier read, decides. Loser gets `SeatUnavailableException`.

Database.java

BookingRepository.java

SeatAlreadyBookedException.java (new)

SeatBookingService.java

VERIFIED – MVN TEST 10/10, 3× MANUAL 4-WAY CONTENTION

T2 Started AI_LOG.md as a per-turn record

One entry per turn (T1, T2, ...) tracking task summary and files touched — backfilled for T1, then itself.

AI_LOG.md (new)

T3 Read-only forensic trace of the race window

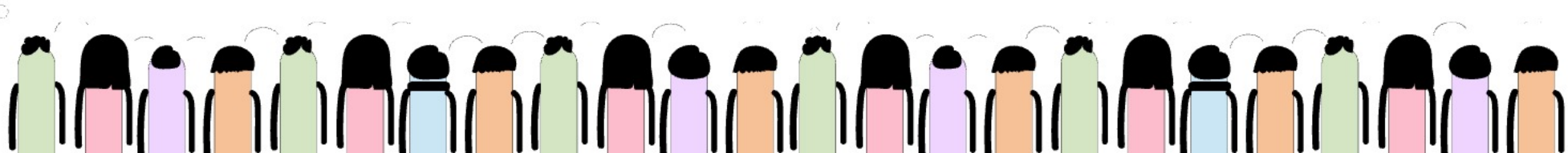
Traced both states — original (2 committed rows) and post-fix (loser rejected, same window) — with file/line evidence, each finding marked **CONFIRMED**. Confirmed by `grep` that `SeatBookingService` is the only caller into the repository. Explained why no existing test or manual QA run could have caught it (single-threaded tests; serial manual clicks can't overlap). No code changed.

evidence/bookSeat-race-trace.md (new)

REFERENCED EVIDENCE/SUPPORT-ESCALATIONS-2026-08-15.MD NOT FOUND IN REPO

seatbook-java

bus seat booking · fare pricing service



Holding the line once the fix already existed

T4 - T6

STEPS

- T4

Flagged a scope mismatch instead of forcing a fit
Asked to generate an accrual/rounding rewrite spec — but this repo has no accrual, rate, or reconciliation code at all. Grep'd to confirm, then asked the user how to proceed rather than inventing a fit.
`evidence/accrual-rounding-implementation-spec.md (new, generated only)`
`NOT IMPLEMENTED - NO MATCHING MODULE EXISTS HERE`
- T5

Added real regression coverage, touched no production code
Asked to redo the fix/test/review cycle with a "don't modify the fix if already applied" rule. Called out that a genuine RED phase was impossible under that constraint, then added a real concurrency test: 8 threads race one seat via `ExecutorService` + `CountDownLatch`, asserting exactly one success and one DB row.
`SeatBookingConcurrencyTest.java (new, test-only)`
`VERIFIED - MVN TEST 11/11, 0 FAILURES`
- T6

Wrote up the full session narrative
This document — a chronological walkthrough of every step above, separate from the per-turn `AT LOG md` index

NET STATE OF THE REPO

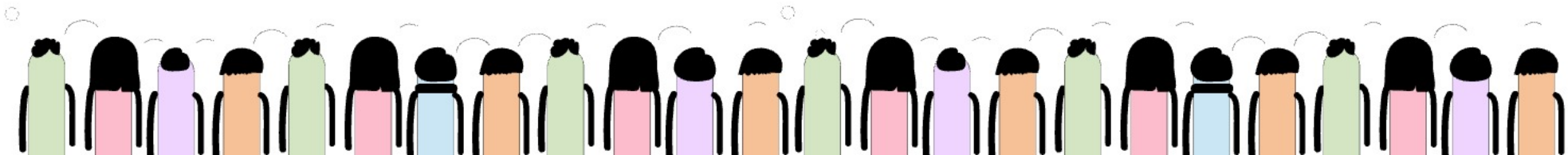
Database.java	CHANGED
BookingRepository.java	CHANGED
SeatAlreadyBookedException.java	NEW
SeatBookingService.java	CHANGED
SeatBookingConcurrencyTest.java	NEW TEST
AI_LOG.md	NEW
bookSeat-race-trace.md	NEW
accrual-rounding-spec.md	NEW

OPEN ITEMS

- support-escalations-2026-08-15.md still not found — fix never correlated to real tickets
- Accrual/rounding spec has no matching module in this repo
- Concurrency fix verified single-JVM/H2 only — no multi-process or production-DB test

seatbook-java

net: 4 files changed · 5 files added · 0 tests broken



Project2



< > **seatbook-java** ☐☐

Name

^

AI_LOG.md

▼ evidence

accrual-rounding-implementation-spec.md

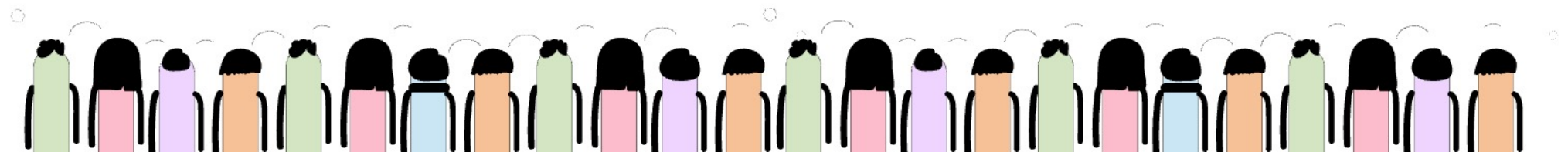
bookSeat-race-trace.md

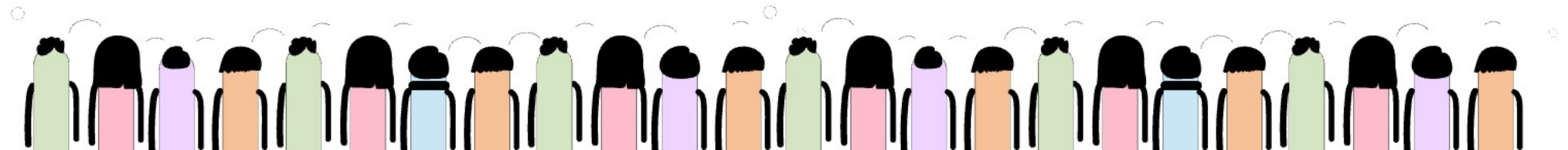
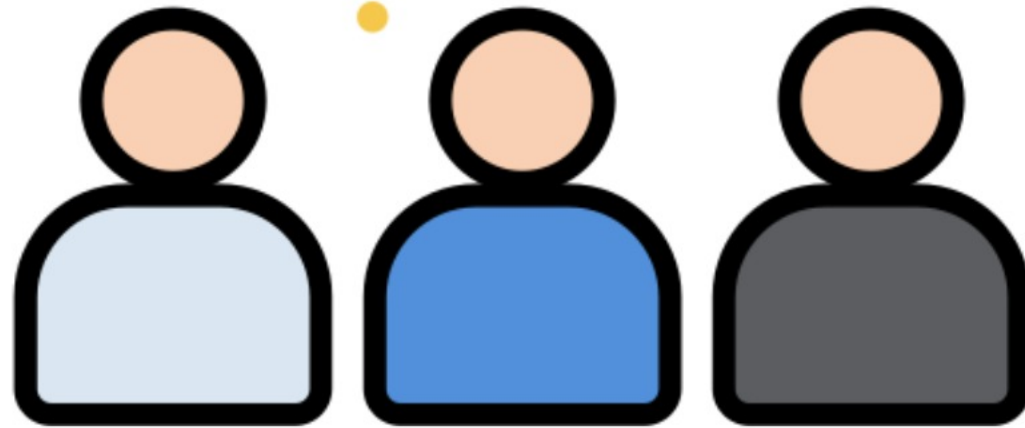
pom.xml

ProjectExecutionSteps.md

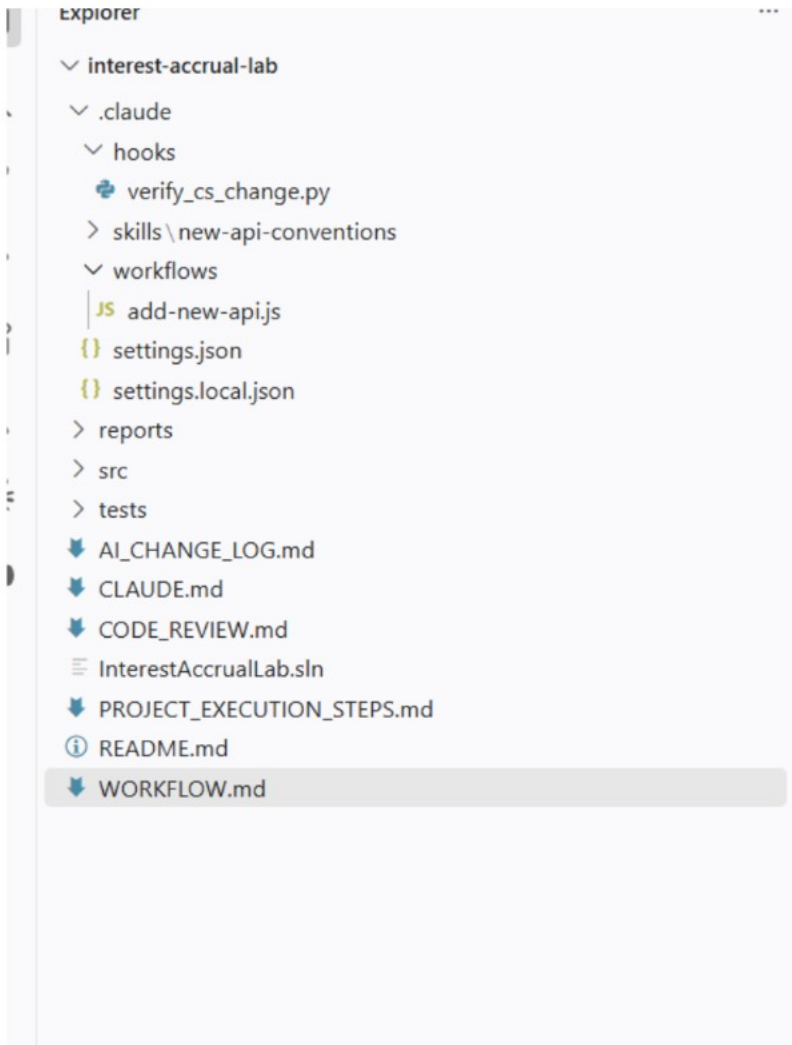
> src

> target





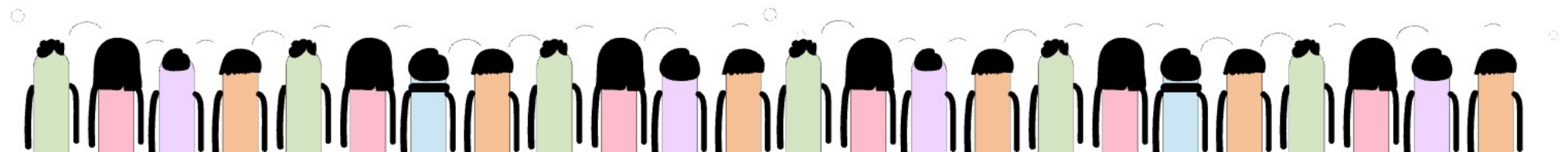
Project3



Hook

Workflow

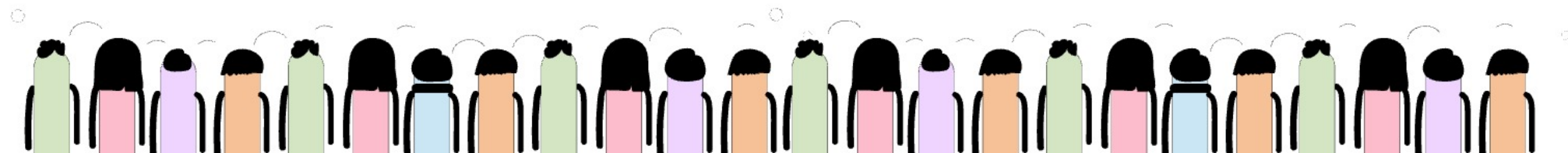
Calude.md



Project3

increasing cost:

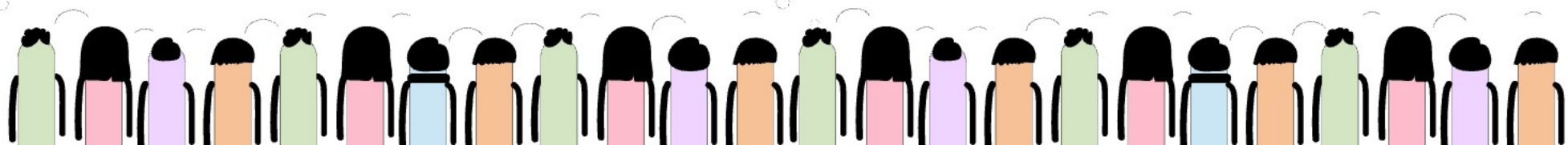
1. **A hook** (`PostToolUse` on `.cs` edits, wired via `/update-config`) — automatically runs `dotnet build` + `dotnet test` after any code change, no prompting needed. This is the direct, cheap fix for the exact 3/5 "verification discipline" gap from the scorecard. Deterministic, near-zero cost, always on.
2. **A named Workflow** for "add a new API end-to-end" — one command that runs the whole pipeline this session did by hand (implement → test-first → self-review against the skill → an *adversarial* compliance-verify pass, structurally, not just when asked → report), the way `/code-review` does with its verify phase. This is the "rich, end-to-end" piece — multiple agents, real token cost, and it's an opt-in tool I can't invoke without you asking for it explicitly.
3. **A scheduled recurring audit** (`/schedule` or `/loop`) — so the 3 open tech-debt items (stale comments, duplicate method, missing midpoint test) get re-surfaced on a cadence instead of just being logged a third time and left, which is what happened this session.

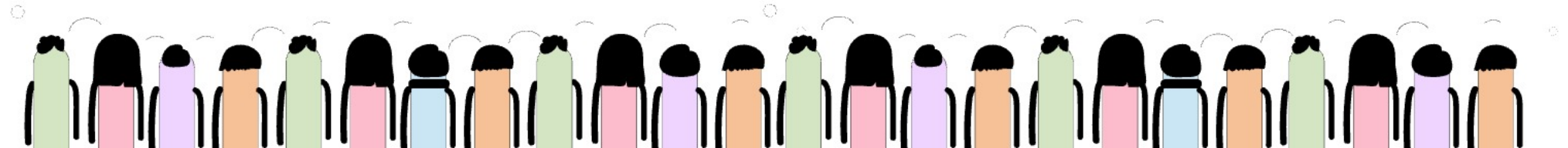
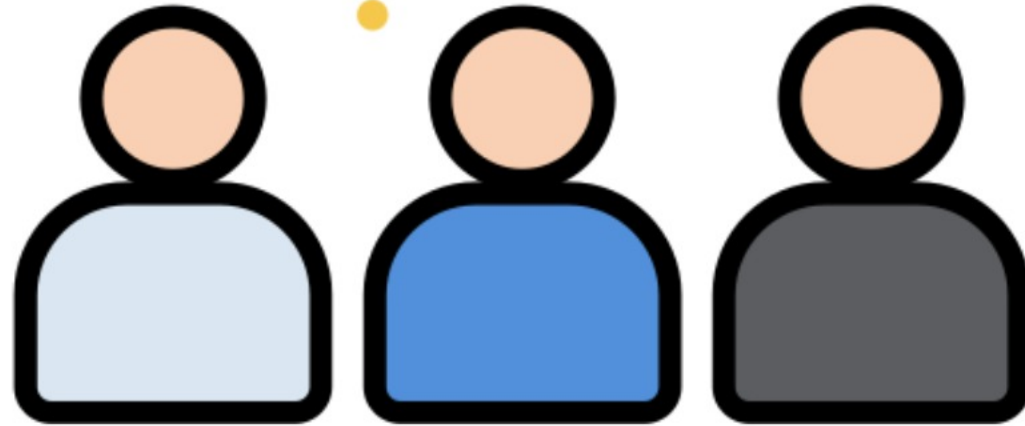


point — it can't be skipped or forgotten.

What happens, concretely, the moment you (or I) save a `.cs` edit:

1. Claude Code invokes `.claude/hooks/verify_cs_change.py`, piping in JSON describing what was edited.
2. The script checks: is this a `.cs` file? If not (e.g. you edit `CLAUDE.md`, like just now), it exits instantly and silently — nothing runs, no cost.
3. If it is a `.cs` file, it identifies which of the four projects (`Banking.Interest`, `Banking.Ledger`, `Banking.Api`, `Banking.Reconciliation`) owns it.
4. It runs `dotnet build` on that project.
5. It runs `dotnet test` on that project's matching test project.
6. Real results come back — a system message you see directly, plus context fed back to me — one of:
 - `tests passed: <project>` (real pass, like `Banking.Api.Tests`)
 - `TESTS FAILED (...)` with actual output, if something's broken
 - `TESTS SKIPPED` (no matching `.NET` runtime installed in this sandbox) for the three `net6.0` projects, which don't have a runtime here — that's an environment gap, not a code failure, and the hook is careful to label it that way rather than reporting a false failure

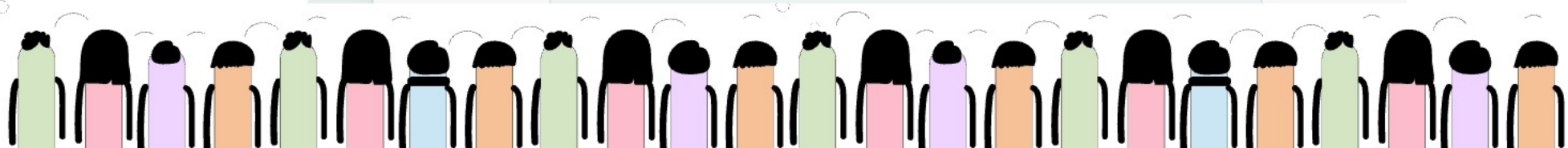
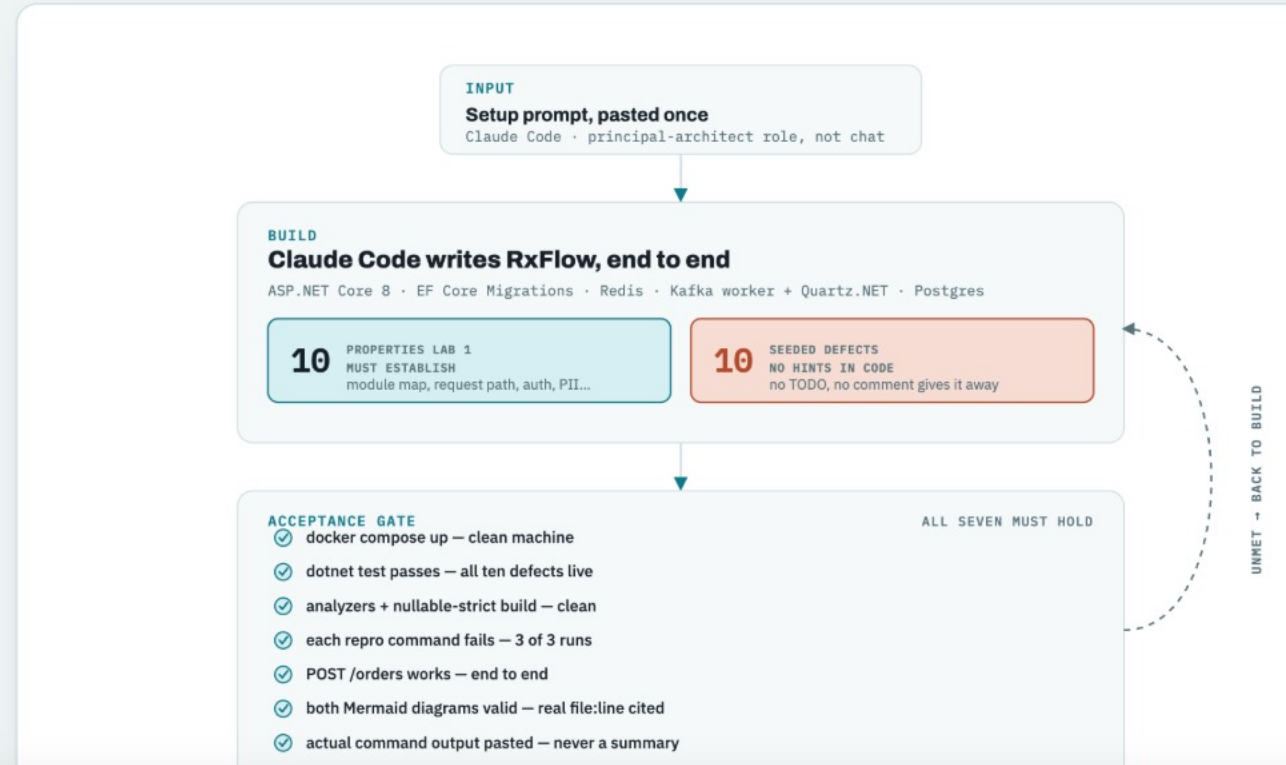




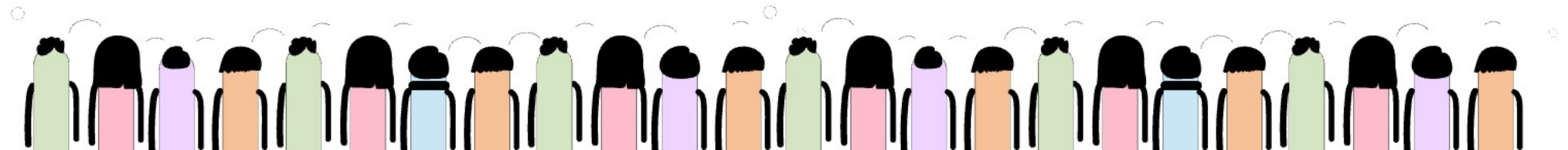
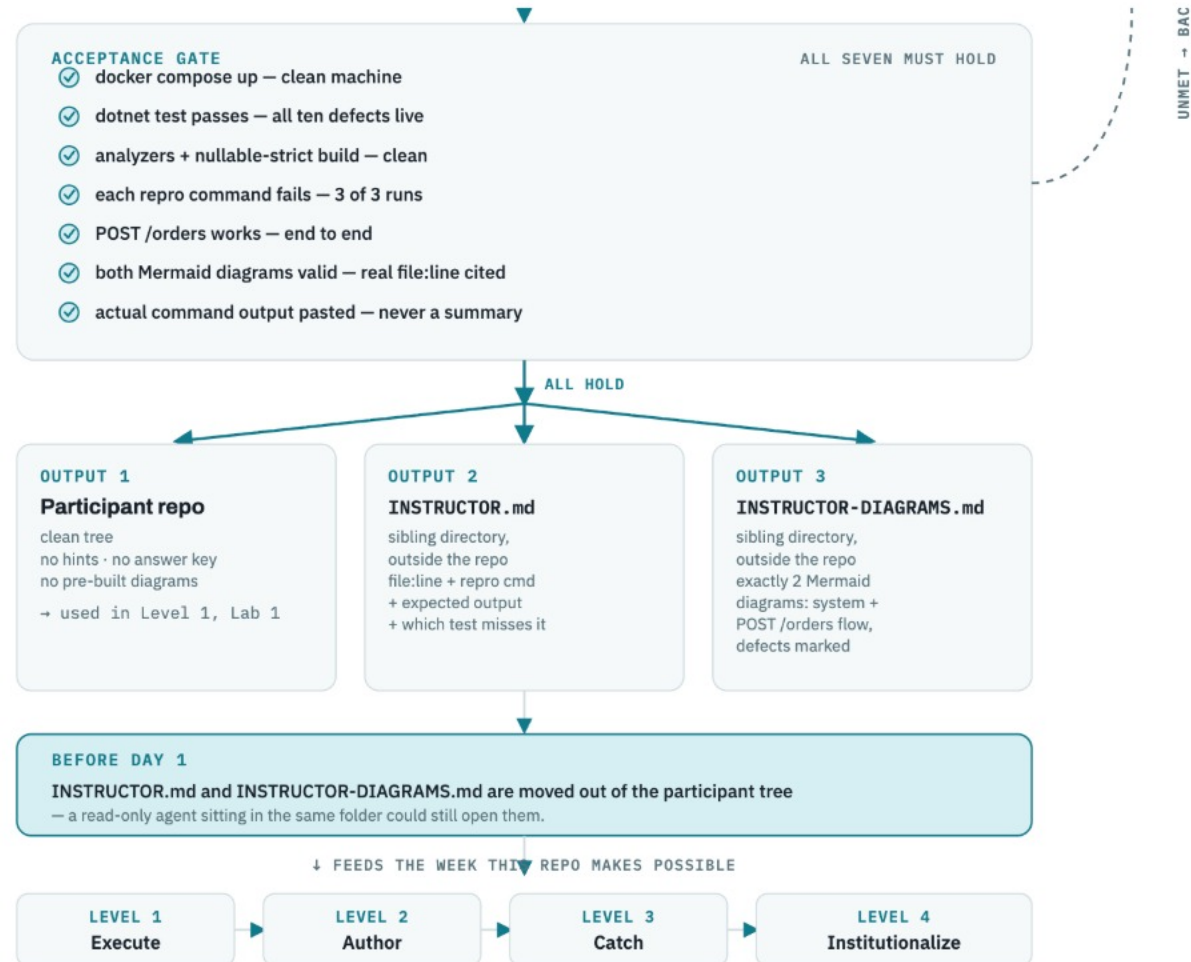
Project4

RxFlow Build Pipeline

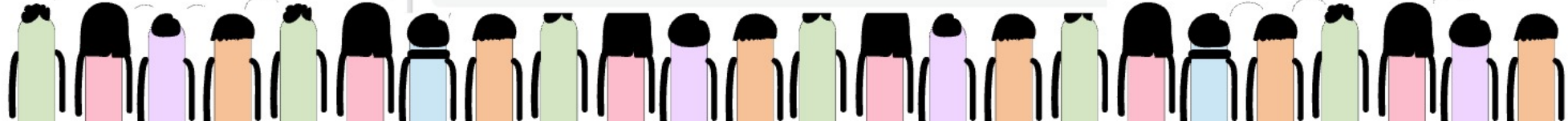
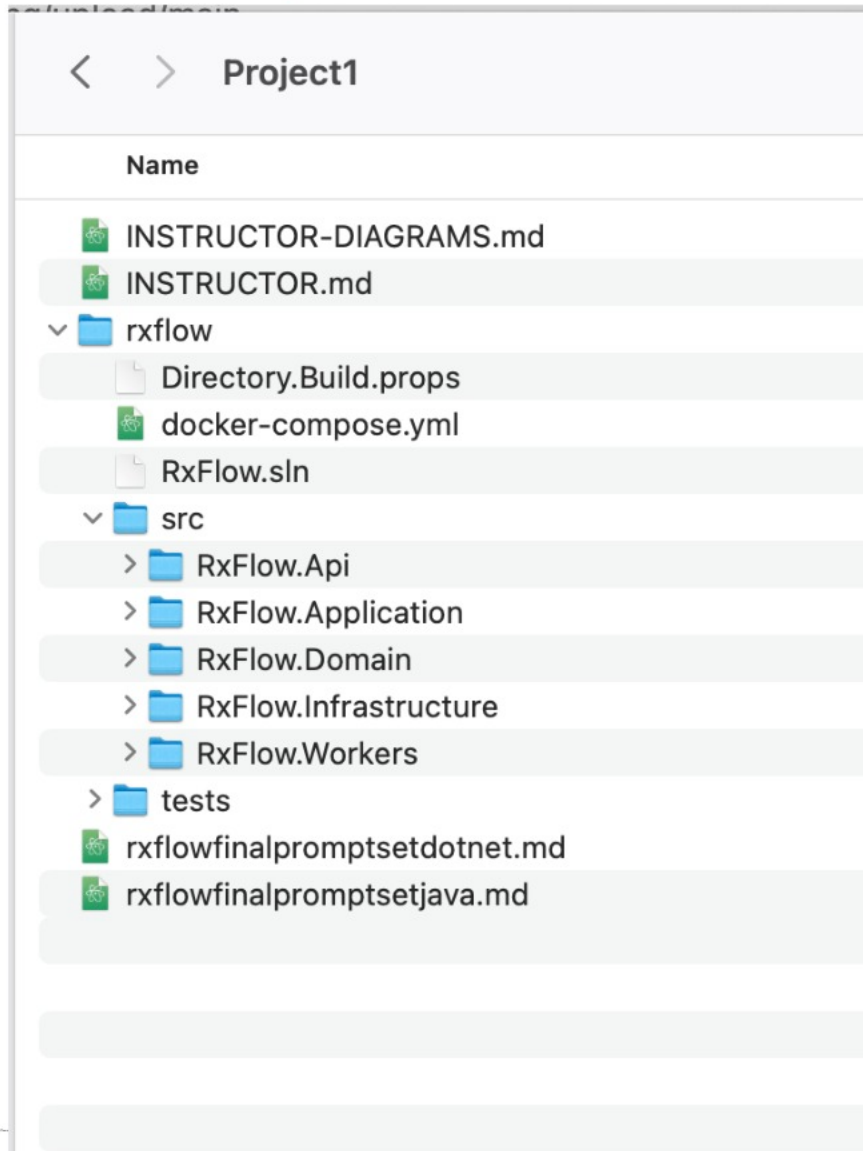
What actually happens when the .NET setup prompt is executed — from build, through the acceptance gate, to its three deliverables and the training week they make possible.

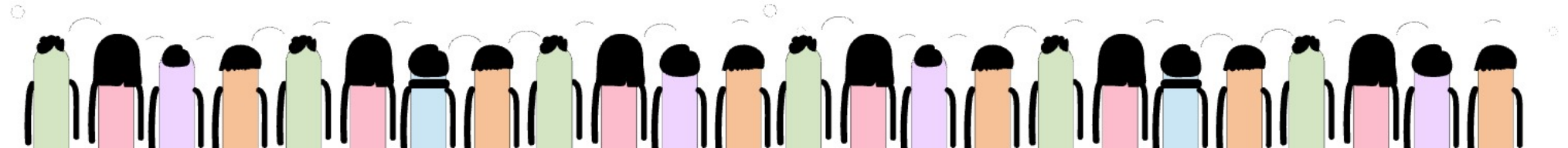
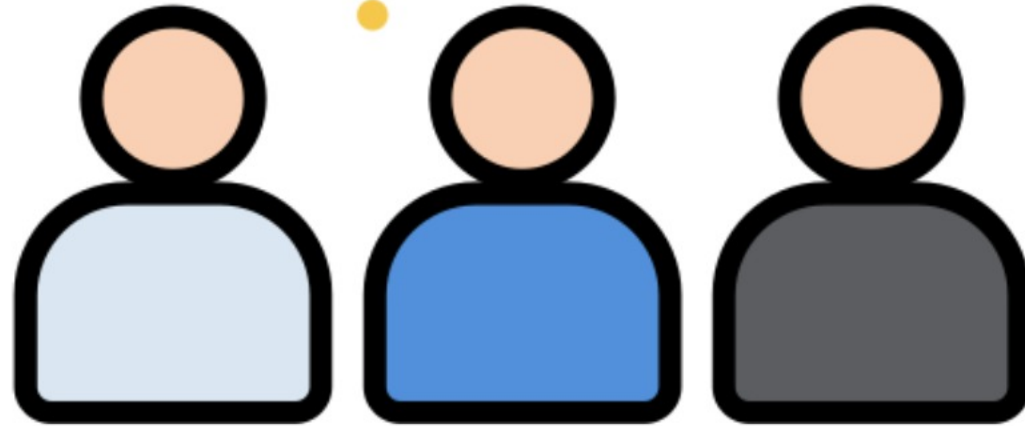


Project4



Project4





Project5

BACKEND ENGINEERING TEMPLATE • FIT RATING

Where the RxFlow-style scaffold actually pays off

01 Legacy code onboarding

BEST FIT

Cold-start into code nobody on the team wrote. Taxonomy sweep and RED→GREEN discipline earn their keep finding **real-shaped bugs** — races, missing idempotency, injection, leaked secrets.

02 Enhancement workflow

GOOD FIT

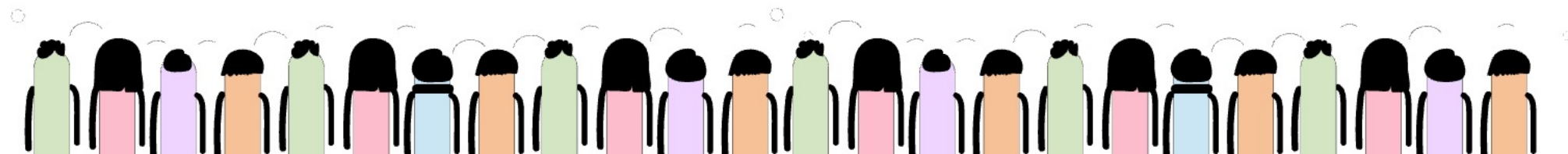
Plan → TDD → self-review → verify, building new features correctly. Leans on standards a bug-fixing pass already **established** — weaker run in isolation.

03 Known codebase bug-fixing

WEAK FIT

You already know the bug and own the code. Taxonomy sweep and cold-start intelligence become **ceremony** — the heavy tooling has little left to discover.

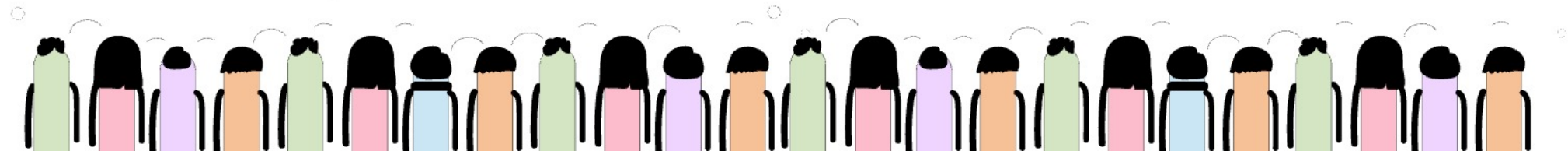
Scaffold: CLAUDE.md • Skill • slash command • hook • sub-agent • MCP • headless CI

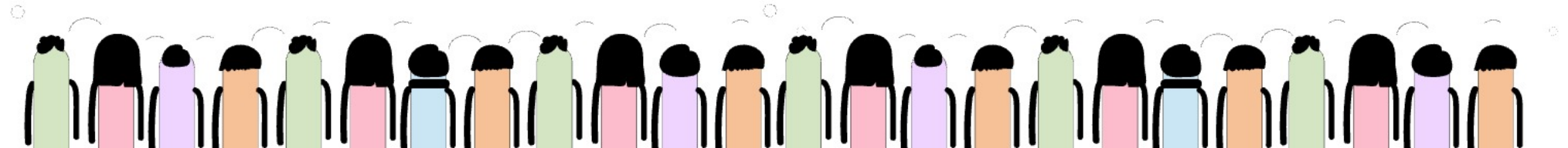
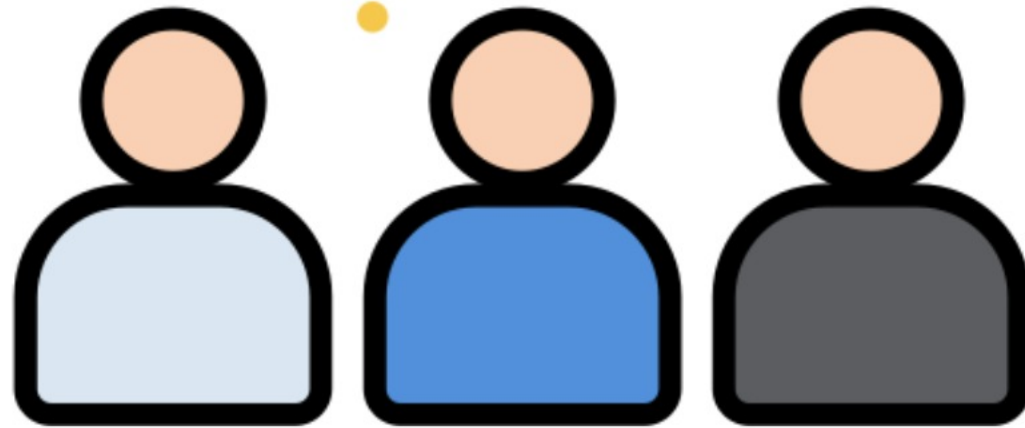


Project5



< > Day5-Prodproject 	
Name	
	CLAUDE.md
	docker-compose.yml
> 	order-service
▼ 	prompts
	DEFECTS_ANSWER_KEY.md
	phase1-skeleton-and-standards.md
	phase2-domain-and-seeded-defects.md
	phase3-cross-cutting-infra-and-analytics.md
	phase4-quality-gates-and-ci.md
	phase5-claude-workflow-tooling.md
	phase6-defect-sweep-execution-and-diagram.md
	phase7-enhancement-workflow.md
	phase8-load-testing-and-wrapup.md
	phase9-skill-vs-slash-command-fix.md

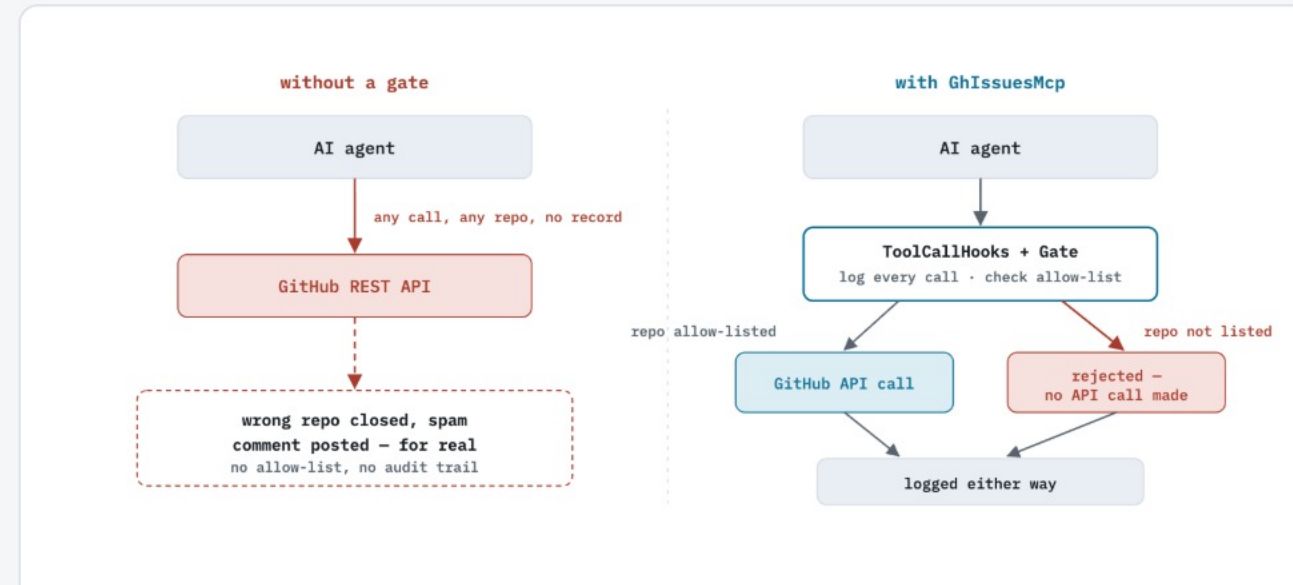




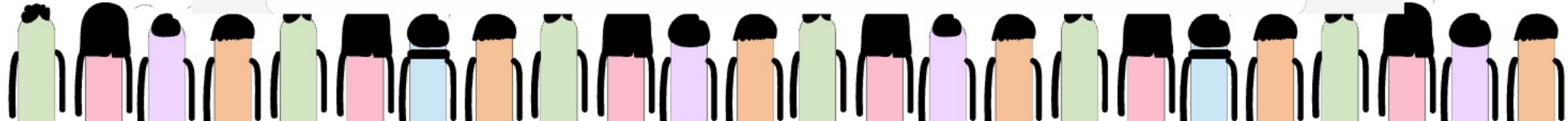
Project6

Why It's Safer

Point an AI agent straight at GitHub's API and every mistake it makes — a wrong repo, a bad instruction, a hallucinated tool call — becomes a real, permanent write. Put it behind `GhIssuesMcp` instead, and the agent can only read freely; anything that writes has to clear an allow-list first, and every attempt — approved or not — leaves a log line.



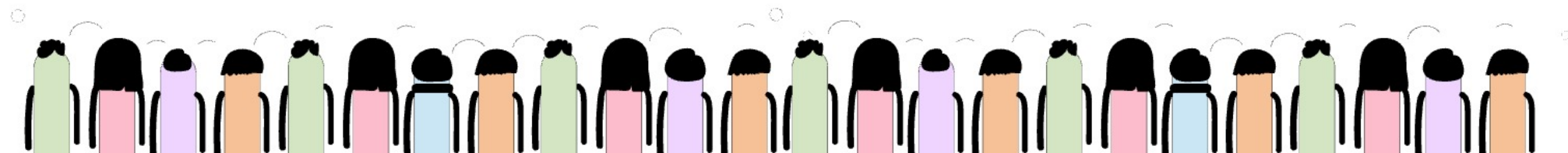
The difference: on the left, the agent's call and GitHub's API sit right next to each other — one bad instruction is one real mutation. On the right, every path — approved or rejected — passes through the same hook, so a mistake becomes a log line instead of a closed issue or a stray comment.



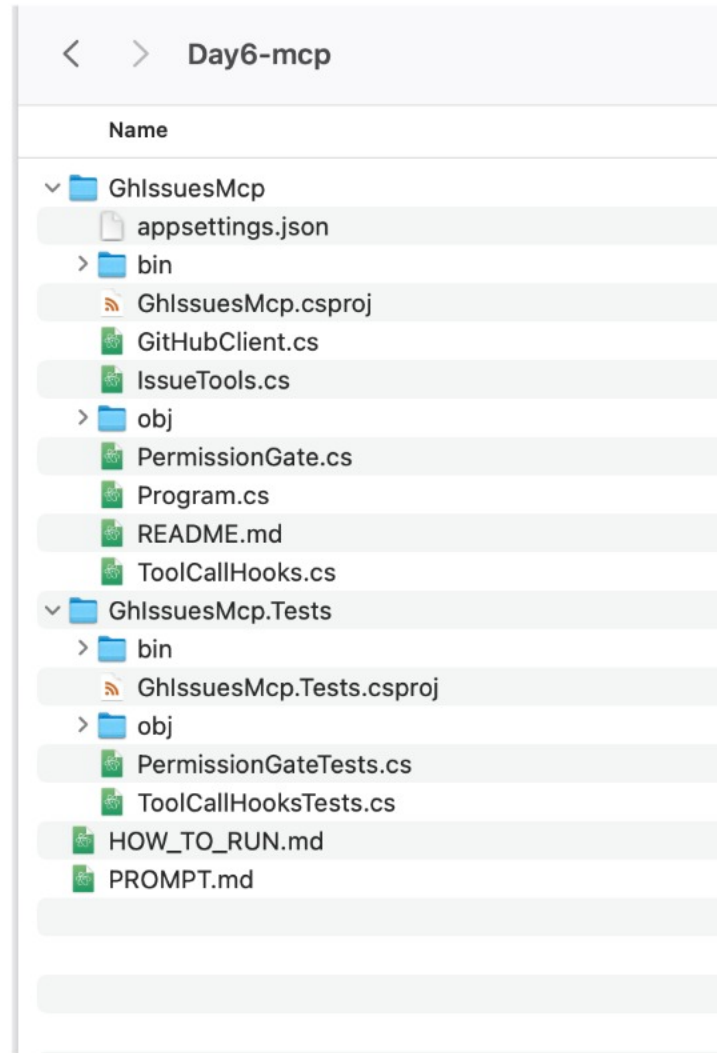
Project6



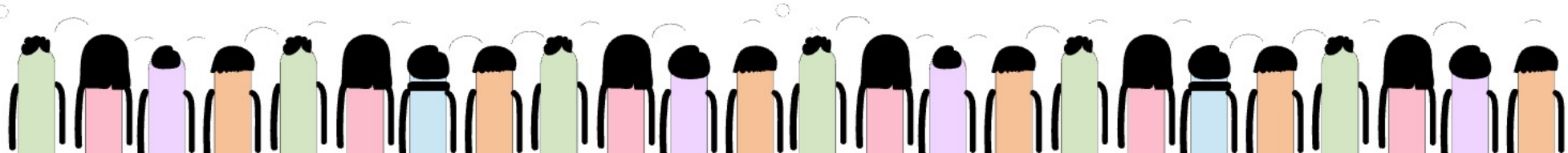
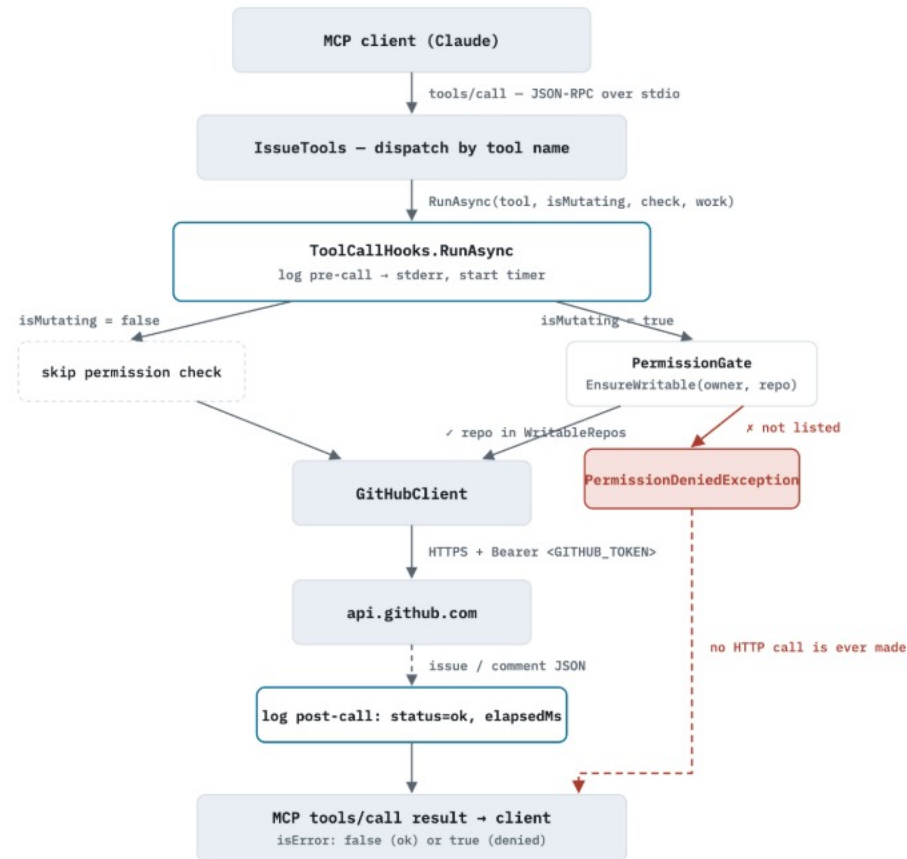
Every tool call — read or write — passes through the same `ToolCallHooks.RunAsync` choke point before it can touch GitHub. That single path is what makes the two guarantees possible: every call gets logged, and a mutating call gets stopped before any HTTP request exists if its repo isn't on the allow-list. The diagram below traces one request from the MCP client down to `api.github.com` and back — and shows exactly where the denied path breaks off.

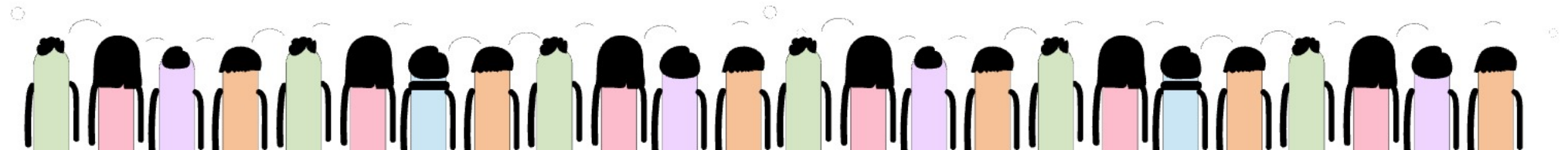
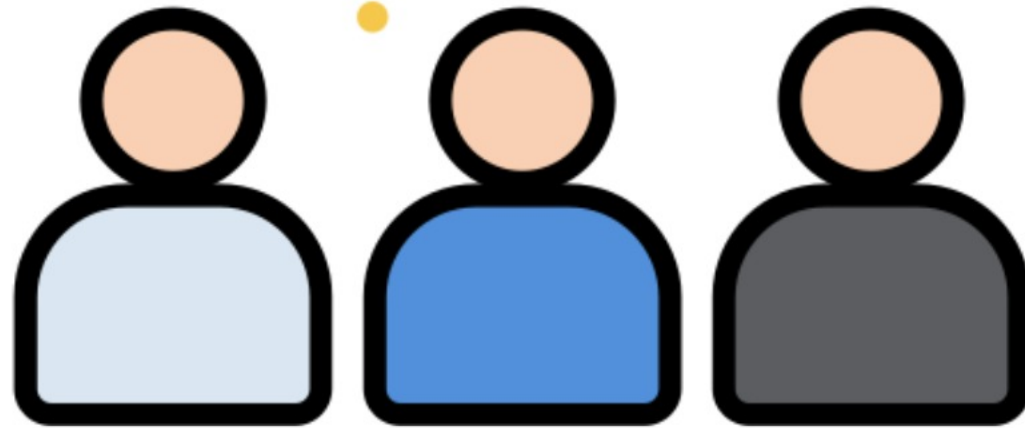


Project6



Project6

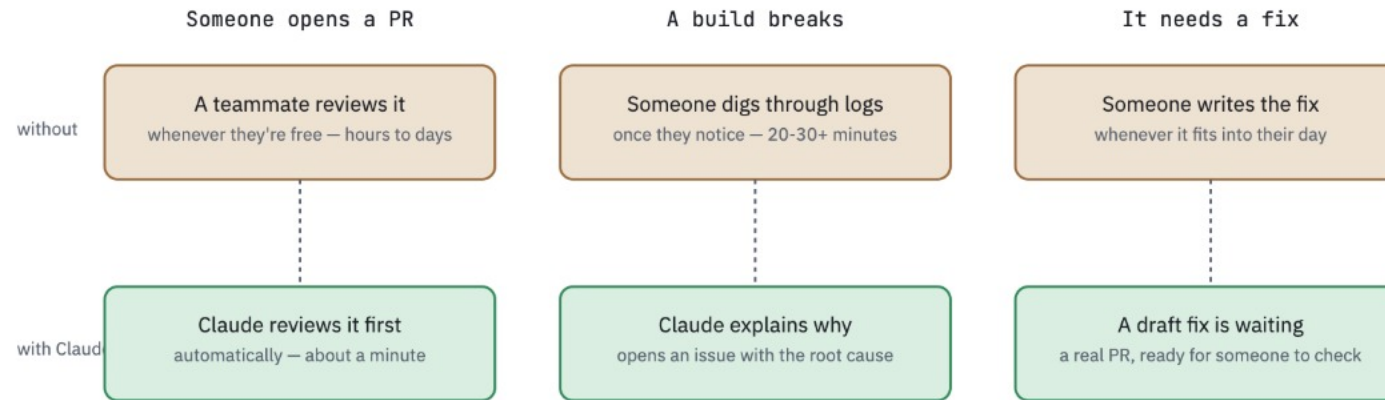




Project7

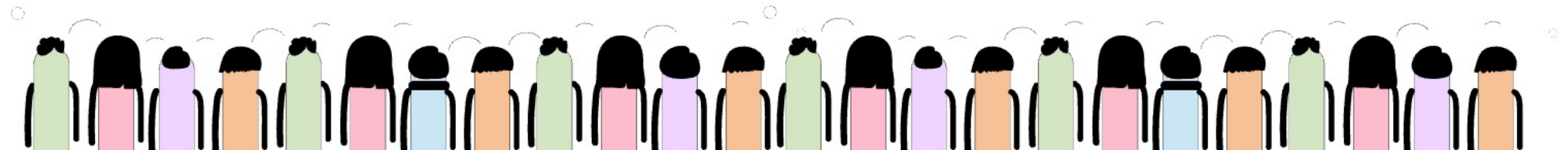
0 – The benefit, in plain terms

■ without this – a person has to do it ■ with this – Claude does it first



A person still decides what merges — Claude just makes sure nothing waits on a person to start.

The benefit isn't "AI writes the code." It's that the slow part of every one of these moments — waiting for a person to have time to look — gets a head start from Claude, while the decision that actually matters (does this merge?) stays with a person.



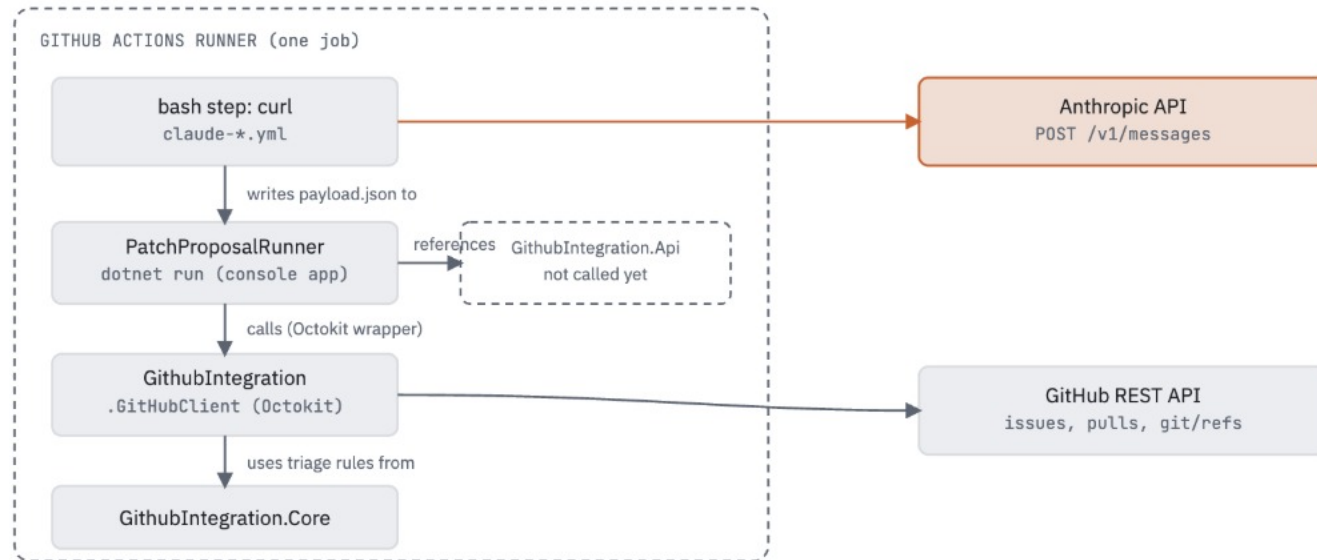
Project7

< > Day7-cicd	
Name	
	CONVERSATION_LOG.md
	diagrams.html
	DotnetClaudeCicd.sln
	EXECUTION.md
	README.md
	SETUP.md
▼ 	src
> 	GithubIntegration.Api
> 	GithubIntegration.Core
> 	GithubIntegration.GitHubClient
▼ 	tests
> 	GithubIntegration.Tests
▼ 	tools
> 	PatchProposalRunner

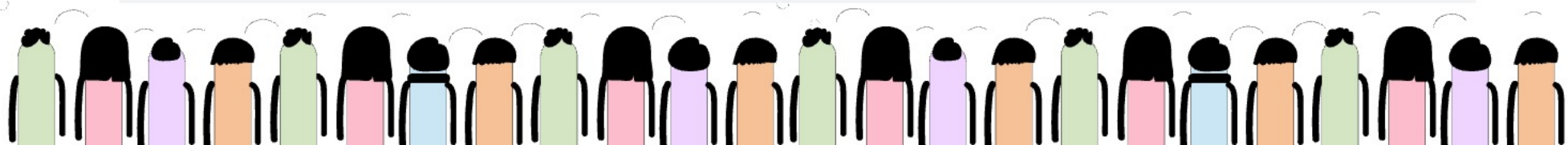


Project7

1 – Where each piece actually runs

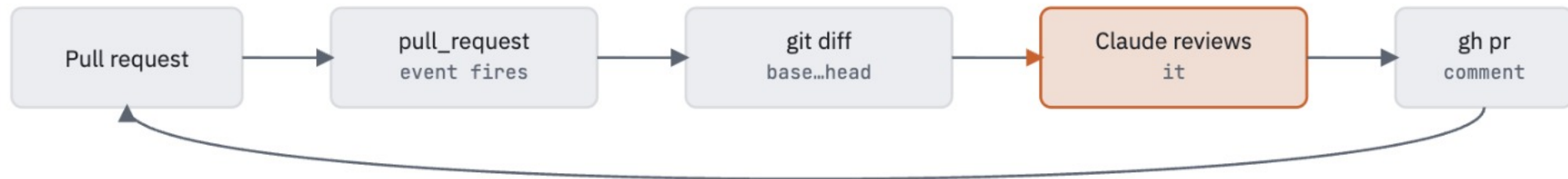


The Claude call and the GitHub call are two separate hops, in two different languages, inside the same job: a bash step talks to Anthropic directly over HTTP, then hands the result to a .NET console app that talks to GitHub through Octokit. `GithubIntegration.Api` is scaffolded but nothing in the pipeline invokes it yet.



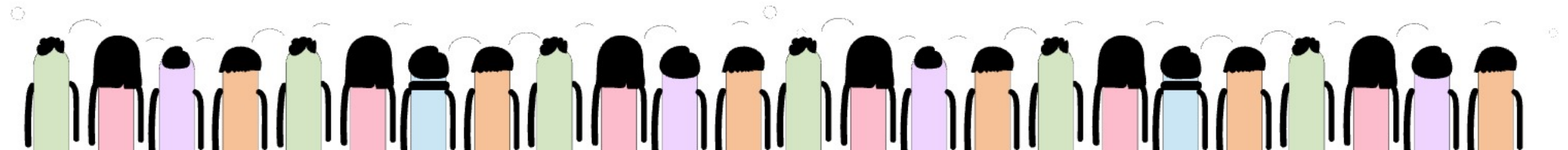
Project7

2 – A pull request gets a real review comment



comment appears on the same PR, usually inside a minute

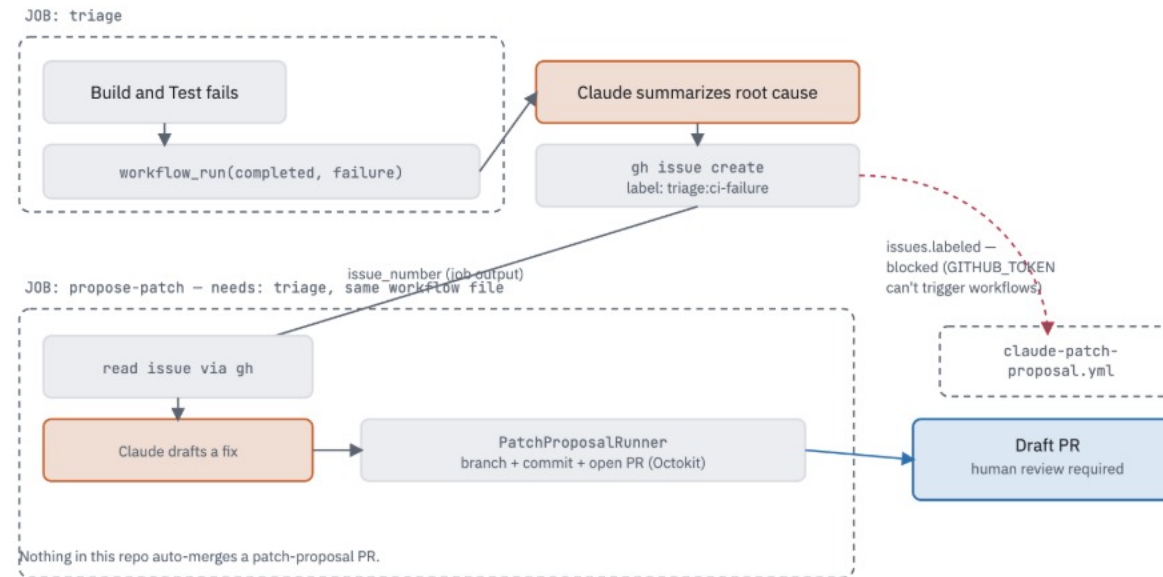
One straight line, no branching: every open or updated PR against `main` gets diffed and reviewed the same way, and the review lands back on the PR it came from.



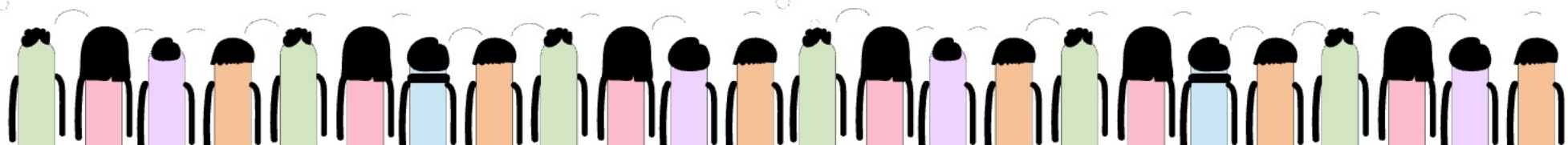
Project7

3 – Why triage and patch-proposal live in one workflow, not two

■ blocked path – never fires ■ human review gate



The obvious design – a second workflow that listens for the `triage:ci-failure` label – silently never runs, because GitHub won't let a workflow's own `GITHUB_TOKEN` activity (like the label added at issue-creation time) start another workflow; that's anti-recursion protection, not a bug. The fix was chaining `propose-patch` directly after `triage` inside one workflow file, passing the new issue's number as a job output. The label-triggered workflow stays wired up for the one case it does work: a human manually labeling an issue.

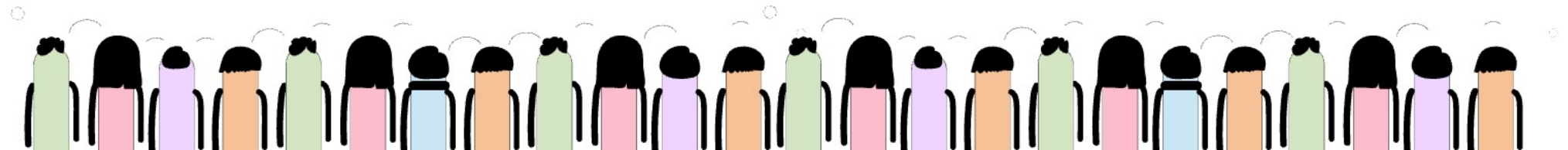


Project7



✓ Fully verified end-to-end:

1. **PR review** — [PR #2](#): `claude-pr-review.yml` ran successfully, Claude posted a real, substantive review comment (caught a genuine bug in the `"Unauthorized"` regex pattern + flagged missing test cases).
2. **CI-failure triage** — [Issue #4](#): a real build failure (`ThisMethodDoesNotExist()`) was caught, Claude correctly diagnosed the exact root cause, and a labeled issue was opened automatically.





TESTIMONIALS

